

Prejšnjič

- niti
 - razred Thread, vmesnik Runnable
 - start() vs. run()
- sinhronizacija niti (synchronized)
 - wait() in notify() le v synchronized blokih
- java.net
 - InetAddress, URL, URI, ServerSocket, Socket

1

```
public class Student extends Thread {  
    private String ime;  
    private int max;  
    private int casPitja;  
  
    public Student(String ime, int max, int casPitja) {  
        this.ime = ime;  
        this.max = max;  
        this.casPitja = casPitja;  
    }  
  
    @Override  
    public void run() {  
        for (int i=1; i<= max; i++) {  
            System.out.println(ime + " pijucka " + i + ". pivce");  
        }  
    }  
}
```

```
public class Abstinent implements Runnable {  
    String kajPije;  
    String ime;  
  
    public Abstinent(String kajPije, String ime) {  
        this.kajPije = kajPije;  
        this.ime = ime;  
    }  
}
```

```
@Override  
public void run() {
```

```
Abstinent marjan = new Abstinent("Marjan", "Cocta");  
new Thread(marian).start();
```

2

```

new Thread(new Runnable() {

    @Override
    public void run() {
        for (int i=0; i<100000000; i++) {
            System.out.println("Anonimni ga šele pije: " + i + "flašo");
            if (i%100==0)
                Thread.currentThread().yield();
        }
    }
}).start();

Thread natakarak = new Thread() {

    @Override
    public void run() {
        while (true) {
            System.out.println("Natakarak zbere narocila");
            try {
                Thread.currentThread().sleep(3000);
            } catch (InterruptedException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            } //catch
        } // while
    } // run
};

natakarak.setDaemon(true);
natakarak.start();

```

3

Niti

Thread, vmesnik Runnable

zagon s start()

- niti in sinhronizacija pri dostopu do deljenih virov
synchronized, wait, notify

```

class Stevec {

    int vrednost;

    void povecaj() {

        synchronized (this) {
            vrednost++;
        }
    }
}

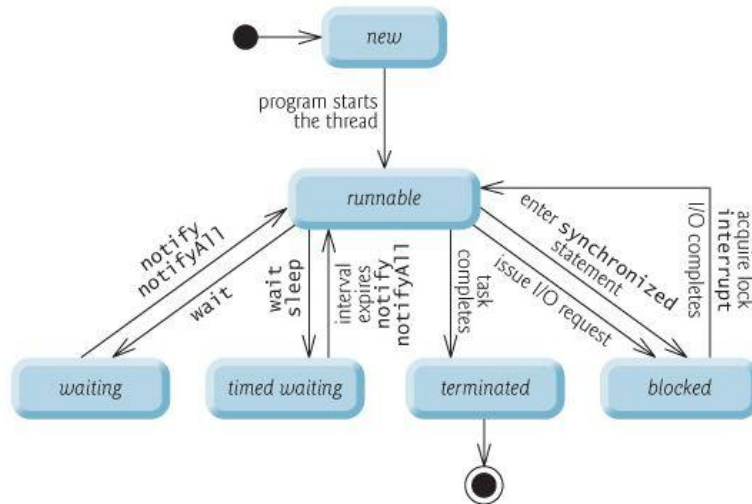
```

4

<https://howtodoinjava.com/java/multi-threading/java-thread-life-cycle-and-thread-states/>

Thread.State

- NEW
- RUNNABLE
- BLOCKED
- WAITING
- TIMED_WAITING
- TERMINATED



5

java.net

Socket
ServerSocket

6

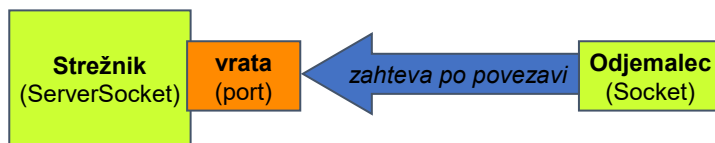
Kaj je Socket

- Protokol TCP omogoča zanesljivo "point-to-point" komunikacijo.
 - Odjemalec in strežnik vzpostavljata povezavo drug na drugega.
 - Vsak program (odjemalec in strežnik) nameni za tako komunikacijo posebno priključno točko – SOCKET (vtičnico)
 - SOCKET predstavlja eno stran dvo-smerne povezave.
-
- Strežnik - razred `java.net.ServerSocket`
 - Odjemalec - razred `java.net.Socket`

7

Vzpostavljanje povezave (1)

- Server ima rezervirana vrata za poslušanje zahtev odjemalcev

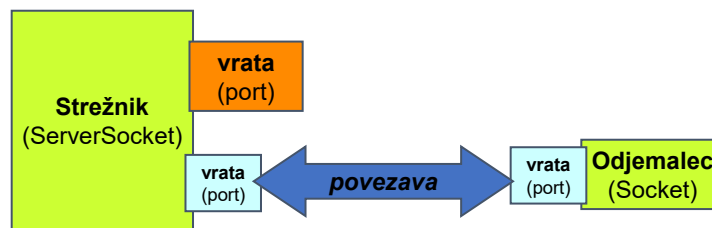


- Odjemalec pozna naslov strežnika in vrata

8

Vzpostavljanje povezave (2)

- Strežnik sprejme zahtevo odjemalca in vzpostavi povezavo.
- Ob sprejetju se na strežniku odprejo nova vrata (Socket), ki je posvečen le določenemu odjemalcu. Prav tako se odprejo vrata na odjemalcu.
- Osnovna vrata so namenjena le za sprejemanje zahtev odjemalcev



9

Pošiljanje/sprejemanje podatkov

- Primer strežnika

```
try {  
    ServerSocket server = new ServerSocket (4444);  
    Socket vticnica = server.accept();  
  
    BufferedReader sprejem = new BufferedReader(  
        new InputStreamReader ( vticnica.getInputStream()  
    ) );  
    BufferedWriter oddaja = new BufferedWriter(  
        new OutputStreamWriter( vticnica.getOutputStream()  
    ) );  
    ...  
    sprejem.close(); oddaja.close();  
    vticnica.close(); server.close();  
} catch (IOException e) {  
    System.out.println("Tezave pri delu s tokovi");  
}
```

vrata (port)

10

Pošiljanje/sprejemanje podatkov

- Primer odjemalca:

```
try {
    Socket vticnica = new Socket("localhost", 4444);

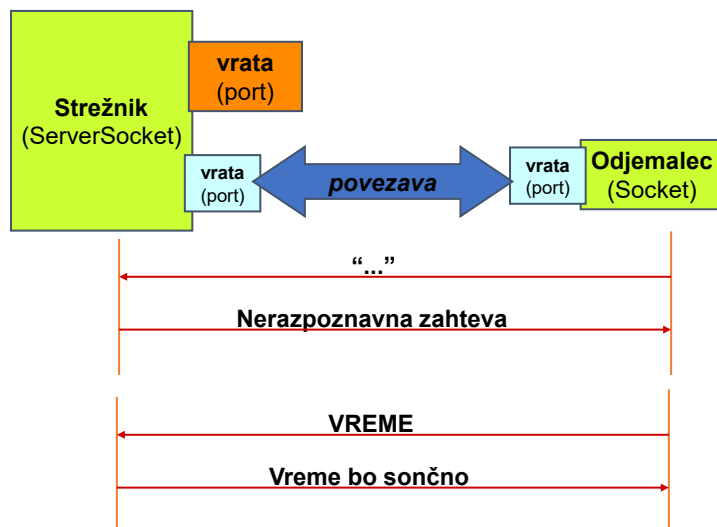
    BufferedReader sprejem = new BufferedReader(
        new InputStreamReader( vticnica.getInputStream() ) );
    BufferedWriter oddaja = new BufferedWriter(
        new OutputStreamWriter( vticnica.getOutputStream() ) );
    ...
    sprejem.close(); oddaja.close(); vticnica.close();
    konzola.close();
} catch (UnknownHostException e) {
    System.out.println("Nezna naslov streznika");
} catch (IOException e) {
    System.out.println("Tezave pri delu s tokovi");
}
```

ime strežnika

vrata (port)

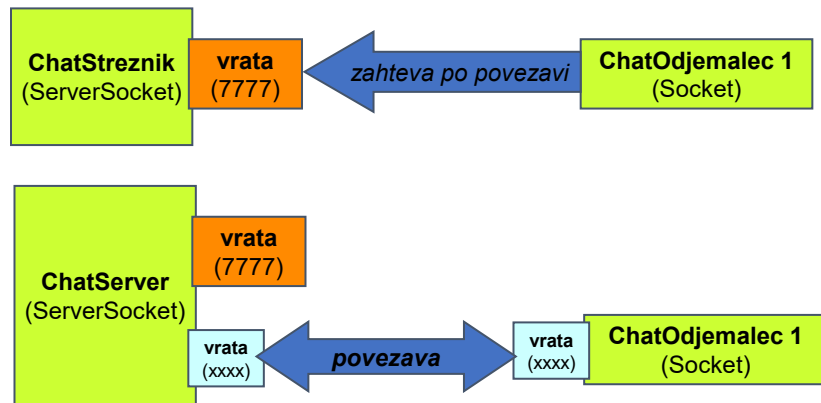
11

Primer



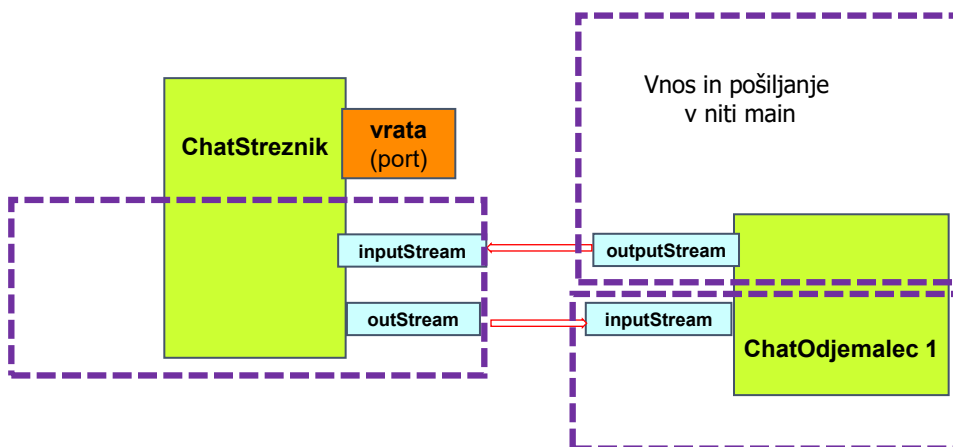
12

Klepetalnica

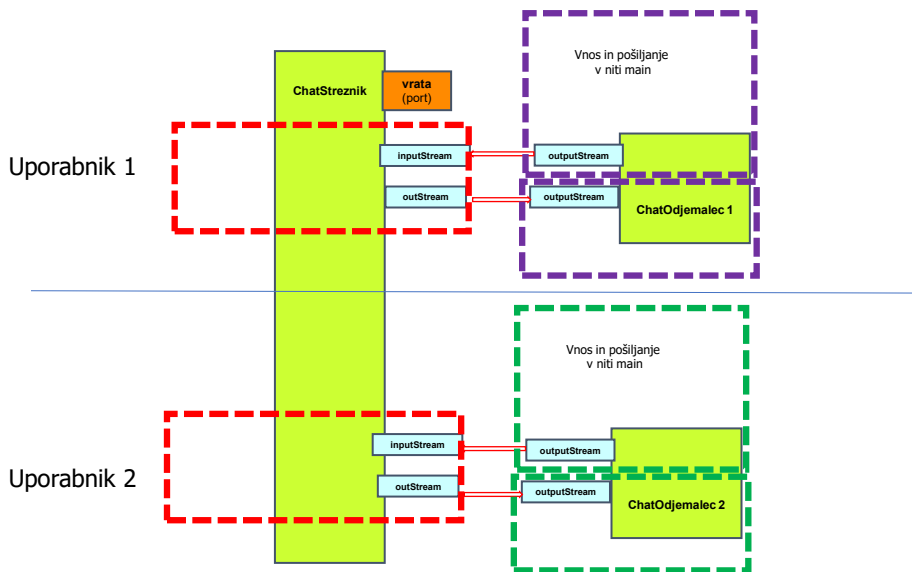


13

Klepetalnica



14



15

Razred **File**

- omogoča delo z zbirkami (razen branja in pisanja)
- ohranja neodvisnost od operacijskega sistema
- ni metode za kreiranje zbirke - v ta namen uporabimo **FileWriter**

Metode za delo z zbirkami

separator	separatorChar
getPath	getAbsolutePath
canRead	isFile
length	mkdir
delete	hashCode
getName	canWrite
pathSeparator	pathSeparatorChar
getParent	exists
isDirectory	isAbsolute
renameTo	makedirs
equals	toString
lastModified	list

16

Java.sql

17

Dostop do baze

java.sql

- potrebujemo JDBC gonilnik (obstaja več tipov)
- JDBC – Java Database Connectivity

18

Dostop do podatkov v bazi

- Naložimo gonilnik za podatkovno bazo

```
Class.forName("gonilnik");
```

- Vzpostavimo povezavo s podatkovno bazo

```
Connection povezava =  
DriverManager.getConnection("povezovalni_niz");
```

- Ustvarimo stavek

```
Statement stavek=povezava.createStatement();
```

- Izvedemo povpraševanje

```
ResultSet rs = stavek.executeQuery("SQL/SQL-J stavek");
```

19

Obdelava podatkov

- Izluščimo rezultate

```
while (rs.next()) {  
    atribut = rs.get<podatkovniTip>(1);  
    atribut2 = rs.get<podatkovniTip>(2);  
    . . .  
}
```

- Zaključimo delo

```
rs.close();  
stavek.close();  
povezava.close();
```

20

Primer

```
import java.sql.*;

public class Povezava {
    public static void main(String[] args){
        try {
            Class.forName("net.ucanaccess.jdbc.UcanaccessDriver");

            Connection conn=DriverManager.getConnection("jdbc:ucanaccess://C:/marjan/Prva.accdb");
            System.out.println("OK");

            Statement st = conn.createStatement();
            ResultSet rs = st.executeQuery("SELECT * FROM Student");
            while (rs.next()) {
                System.out.print(rs.getObject("ID"));
                System.out.print(" - " + rs.getObject("Ime"));
                System.out.println(" - " + rs.getObject(3));
                System.out.println(" - " + rs.getObject(4));
            }
        } catch (SQLException | ClassNotFoundException e) {
            System.out.println("fail");
            e.printStackTrace();
        }
    }
}
```

21

Povezovanje s podatkovno bazo

- Naložimo razred gonilnika z uporabo metode `forName()` razreda `Class`.
- Izvedemo povezavo z metodo `getConnection(String url)` razreda `DriverManager`

```
public static Class<T> forName(String className)
    throws ClassNotFoundException {
    Returns the Class object associated with the class or interface with the given string name. Invoking this method is equivalent to:
    Class.forName(className, true, currentLoader)
    where currentLoader denotes the defining class loader of the current class.
    For example, the following code fragment returns the runtime Class descriptor for the class named java.lang.Thread:
    Class t = Class.forName("java.lang.Thread")
    A call to forName("X") causes the class named X to be initialized.
    In cases where this method is called from a context where there is no caller frame on the stack (e.g. when called directly from a JNI attached thread), the system class loader
    is used.
    Parameters:
    className - the fully qualified name of the desired class.
    Returns:
    the Class object for the class with the specified name.
    Throws:
    LinkageError - if the linkage fails
    ExceptionInInitializerError - if the initialization provoked by this method fails
    ClassNotFoundException - if the class cannot be located
```

22

Connection Interface

Pomembne metode

- `close()` - zapre povezavo do podatkovne baze
- `getMetaData()` - vrne objekt, ki vsebuje podrobne informacije o podatkovni bazi na katero smo povezani
- `createStatement()` - oblikuje objekt z SQL stavkom
- `prepareStatement()` - podobno kot prejšnja, le da je SQL stavek predhodno obdelan in je zato izvajanje učinkovitejše

23

Izvajanje povpraševanja

Potek izvajanja SQL stavka je sledeč:

- pripravimo si niz v obliki SQL stavka
- pripravimo si objekt tipa `Statement`
- prožimo eno izmed metod za izvršitev stavka, kateri kot parameter podamo oblikovan SQL niz.

24

Vrnjeni podatki (ResultSet)

- Rezultat povpraševanja dobimo v objektu tipa **ResultSet**.
- Z metodo **getMetaData()** dobimo informacijo o zapisih (imena stolpcev, število stolpcev, ipd.).
- Podatke razčlenimo tako, da beremo zapise v zanki in jih izpišemo.

25

Meta-podatki

- Iz same povezave (Connection) lahko pridobimo informacije (meta podatke):
 - Seznam katalogov
 - Seznam shem
 - Seznam tabel
 - ...
- Iz rezultirajoče množice (ResultSet) lahko pridobimo informacije (meta podatke):
 - Seznam stolpcev
 - Podatkovni tipi posameznega stolpca
 - Imena stolpcev
 - Ime tabele
 - ...

26

javax.sql

vmesnik **DataSource** (z metodo getConnection)

Pridobitve:

- **imena ne „zapečemo“ v svojo kodo**
(uporabljamo logična imena)
- **upravljanje in dodeljevanje povezav iz „bazena“**
(večuporabniško in porazdeljeno delovanje – če podpira implementacija)

„sistemski del“ (administratorski)

1. Kreiranje primerka razreda, ki implementira DataSource
2. Nastavitev lastnosti
3. Registracija preko storitev poimenovanja JNDI API
(Java Naming and Directory Interface)

27

Spletne storitve (REST)

28

Primer odjemalca

```
URL url = new URL("http://example.com/customers/1");
connection = (URLConnection) getUrl.openConnection();
connection.setRequestMethod("GET");
connection.setRequestProperty("Accept", "application/xml");
if (connection.getResponseCode() != 200) {
    throw new RuntimeException("Operation failed: "+
        connection.getResponseCode());
}
System.out.println("Content-Type: "+connection.getContentType());
BufferedReader reader = new BufferedReader(
    new InputStreamReader(connection.getInputStream()));
String line = reader.readLine();
while (line != null) {
    System.out.println(line);
    line = reader.readLine();
}
connection.disconnect();
```

29

Uporaba java.net.http

- HttpRequest, HttpClient

```
HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create(baseUrl))
    .header("Accept", "application/json")
    .GET()
    .build();

HttpClient httpClient = HttpClient.newHttpClient();

HttpResponse<String> response =
    httpClient.send(request, HttpResponse.BodyHandlers.ofString(StandardCharsets.UTF_8));

System.out.println(response.body());
```

30

Obdelava

- JSON (JSON-B, Jackson, G-SON)

```
import jakarta.json.bind.Jsonb;
import jakarta.json.bind.JsonbBuilder;

// Objekt -> JSON
String json = jsonb.toJson(JANEZ);
System.out.println("JSON:");
System.out.println(json);

// JSON -> Objekt
Oseba osebaIzJson = jsonb.fromJson(json, Oseba.class);
System.out.println("Nazaj iz JSON:");
System.out.println(osebaIzJson);
```

```
import com.fasterxml.jackson.databind.ObjectMapper;

ObjectMapper mapper = new ObjectMapper();

// ===== Objekt -> JSON =====
String json = mapper.writeValueAsString(JANEZ);
System.out.println("JSON:");
System.out.println(json);

// ===== JSON -> Objekt =====
Oseba osebaIzJson = mapper.readValue(json, Oseba.class);
System.out.println("Nazaj iz JSON:");
System.out.println(osebaIzJson);
```

31

Obdelava XML

```
import jakarta.xml.bind.JAXBContext;
import jakarta.xml.bind.Marshaller;
import jakarta.xml.bind.Unmarshaller;
```

```
import java.io.StringReader;
import java.io.StringWriter;
```

```
// ===== JAXB (XML) =====
JAXBContext context = JAXBContext.newInstance(Oseba.class);

// Objekt -> XML
Marshaller marshaller = context.createMarshaller();
marshaller.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);

StringWriter writer = new StringWriter();
marshaller.marshal(JANEZ, writer);

String xml = writer.toString();
System.out.println("\nXML:");
System.out.println(xml);

// XML -> Objekt
Unmarshaller unmarshaller = context.createUnmarshaller();
StringReader reader = new StringReader(xml);

Oseba osebaIzXml = (Oseba) unmarshaller.unmarshal(reader);
System.out.println("Nazaj iz XML:");
System.out.println(osebaIzXml);
```

32

IBM Visual Age

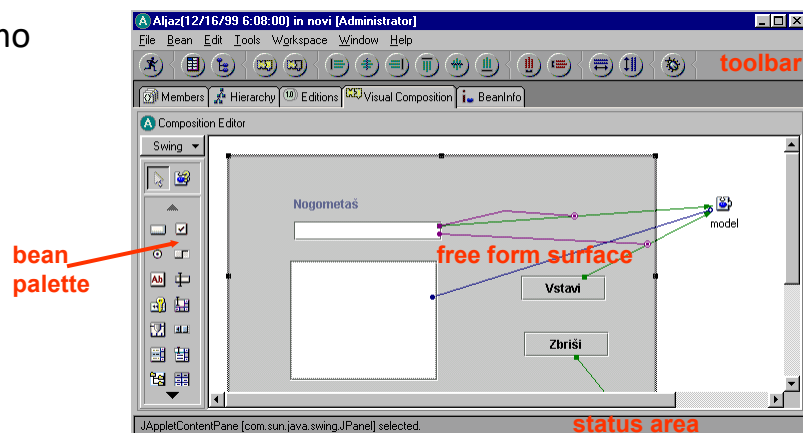
razvojno okolje
vizualno programiranje

33

Vizualno programiranje

- Visual Composition Editor
integriran v IBM VAJa

delamo
z zrni



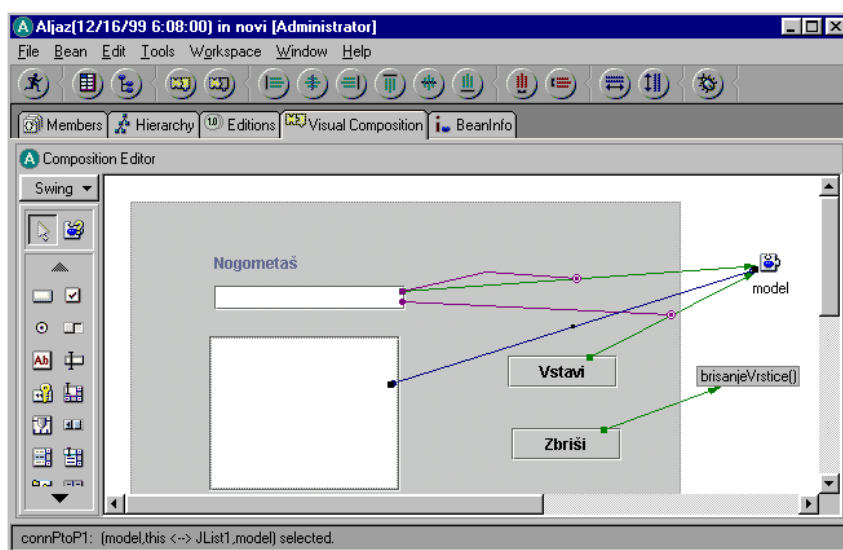
34

Tipi povezav (connections)

- **event-to-method**
povzroči, da se proži metoda, ko se pojavi dogodek (povezava med dvema zrnoma)
- **parameter from property** ali **code** ali **method**
- **property-to-property**
povezuje dve podatkovni vrednosti
- **event-to-code**
požene določeno metodo manipularanega razreda, ko se pojavi dogodek (zrno - composit zrno)

35

Primer



36

Javanska Zrna

Java Beans

37

Komponente

- dedovanje in polimorfizem sta osnovni značilnosti objektne paradigme
- včasih pa bi kljub vsemu želeli neko komponento preprosto uporabiti takšno kot pač je (analogija - sestavljanje komponent v drugih industrijah)
- zgodovina:
UI gradniki: VB, Delphi...

38

V času načrtovanja

- nastavimo lastnosti (properties)
npr. barva, font, baza na katero se poveže...
znotraj razvojnega okolja -
ob zagonu programa te lastnosti veljajo
- dogodki, ki se lahko pojavijo
povežemo z neko programsko kodo

Kako?

Dinamični vpogled z uporabo mehanizma "reflection" - dobimo informacije o lastnostih in dogodkih, ki jih podpira komponenta

39

Java Beans

- **JavaBeans** are classes that encapsulate many [objects](#) into a single object (the bean). They are [serializable](#), have a [zero-argument constructor](#), and allow access to properties using [getter and setter methods](#). The name "Bean" was given to encompass this standard, which aims to create [reusable software components](#) for [Java](#).

40

Java zrna

- Java Beans – Javin/javanski komponentni model
- Java komponenta je preprosto - razred!!
 - ni potrebno pisati posebne kode ali uporabljati posebnih razširitev jezika (včasih za VB in Delphi)
- **potrebno je ustrezno poimenovati metode**
- ime razvojnemu okolju razkrije, ali gre za lastnost, dogodek ali navadno metodo

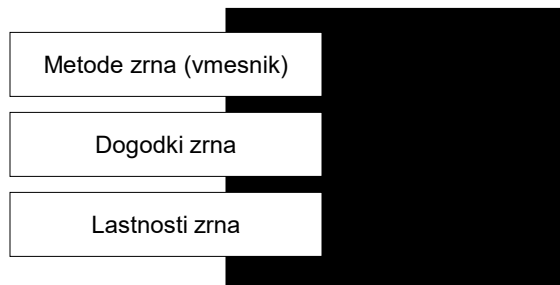
41

Dogovori glede poimenovanja

- za lastnosti
 - **getXxx, setXxx**
 - za logične (boolean) **isXxx**
- za dogodke
 - uporaba pristopa poslušalcev
- navadne metode so **public**

42

Zrno = črna škatla



43

Komponente

- zrno - gradnik pri CBD
- zrno v zrnu v zrnu ...
- ne le vizualna zrna (ta izpeljana iz Component)!
- značilnosti
 - vpogled (introspection)
 - prilagodljivost (customization)
 - komunikacija z dogodki (events)
 - obstojnost (persistence)

44

Skratka

- za vpogled običajno poskrbijo orodja
- Introspekcija oz. vpogled možen
 - nižji nivo – Reflection API
 - meta informacije zrna - BeanInfo
- ni posebnega jezika za opis zrna, držimo pa se pravil getter in setter metode
- zrna se obnašajo po pravilih, ki so zastavljena v Javinem varnostnem modelu

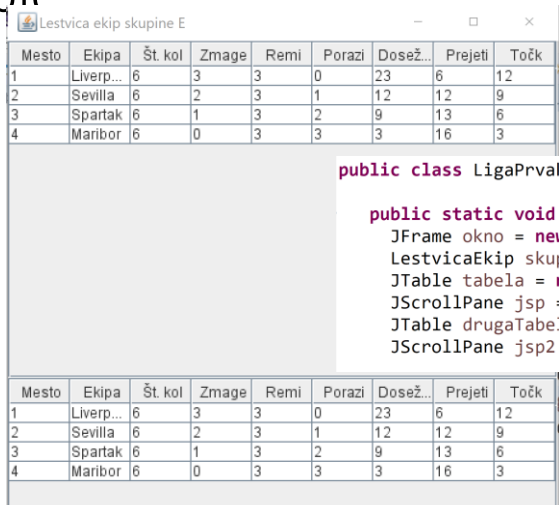
45

Swing komponente so javanska zrna

- nabor prilagodljivih vizualnih gradnikov
- ob vpeljavi del JFC (Java Foundation Classes)
- Ime gradnikov: **J**Gradnik
- ni “peer” razredov na gostujočem sistemu (*lightweight komponente*)
- **možnost zagotovitve enovitega izgleda (L&F) na različnih platformah, privzet izgled: Metal**
- uporaba vzorca MVC (Model View Controler)

46

JTable

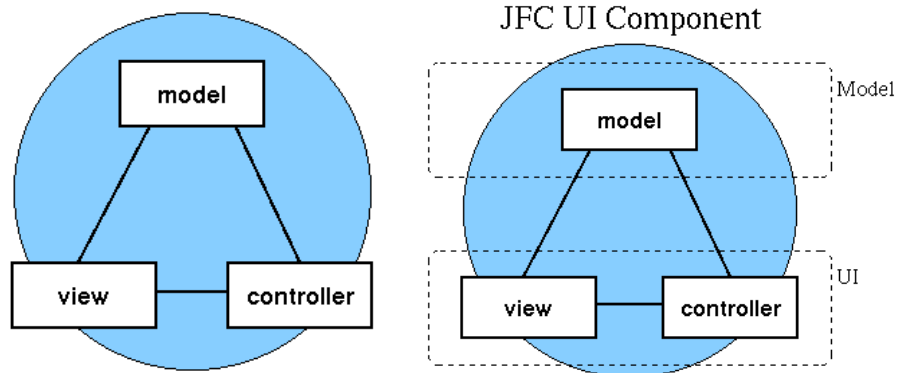


```
public class LigaPrvakov {  
  
    public static void main (String[] args) {  
        JFrame okno = new JFrame("Lestvica ekip skupine E");  
        LestvicaEkip skupinaE = new LestvicaEkip();  
        JTable tabela = new JTable (skupinaE);  
        JScrollPane jsp = new JScrollPane(tabela);  
        JTable drugaTabela = new JTable (skupinaE);  
        JScrollPane jsp2 = new JScrollPane(drugaTabela);  
    }  
}
```

Dve tabeli prikazujeta identične podatke!!!

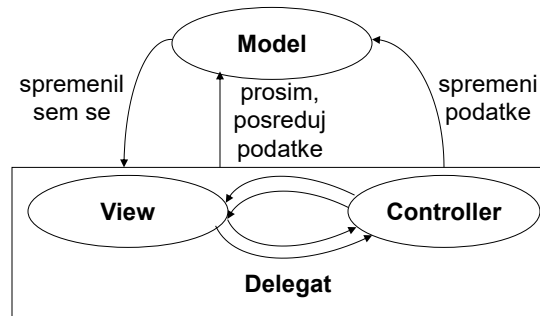
47

Delitev odgovornosti



48

Kombiniran pogled in nadzornik- delegat



Ločitev med pogledom in nadzornikom ni tako pomembna,
kot ločitev med modelom in delegatom!!!

49

Lambda izrazi

Lambda je anonimna funkcija

`() -> {}`

`parameter -> expression`

`(parameter1, parameter2) -> expression`

`(parameter1, parameter2) -> { blok kode }`

50

Funkcijski vmesniki

@FunctionalInterface

natančno ena abstraktna metoda

SAM (Single Abstract Method)

```
@FunctionalInterface
public interface Printable {

    void natisni(String p, String d);

}
```

```
static void poklici(Printable p) {
    p.natisni("spredaj", "zadaj");
}

public static void main(String[] args) {

    Printable x = (s,s1) -> System.out.println(s+ "mijuw" + s1);

    poklici(x);
    poklici((s1,s2)-> System.out.println(s1+" hou hou" + s2));
}
```

51

```
gumb.addActionListener((e)-> System.out.println(e.getSource()));
```

```
gumb.addActionListener(new ActionListener() {
```

```
    @Override
    public void actionPerformed(ActionEvent e) {
        // Kar precej daljše...
        System.out.println(e.getSource());
    }
});
```

52

Poenostavitve

```
ArrayList<Oseba> osebe = new ArrayList<Oseba>();
osebe.add(new Oseba("Janez", 22));
osebe.add(new Oseba("Janezek", 16));
osebe.add(new Oseba("Metka", 22));
osebe.add(new Oseba("Metka", 18));
osebe.add(new Oseba("Marica", 82));
osebe.add(new Oseba("Mojca", 17));
osebe.add(new Oseba("Metka ml.", 15));

osebe.forEach(nekdo -> System.out.println(nekdo));

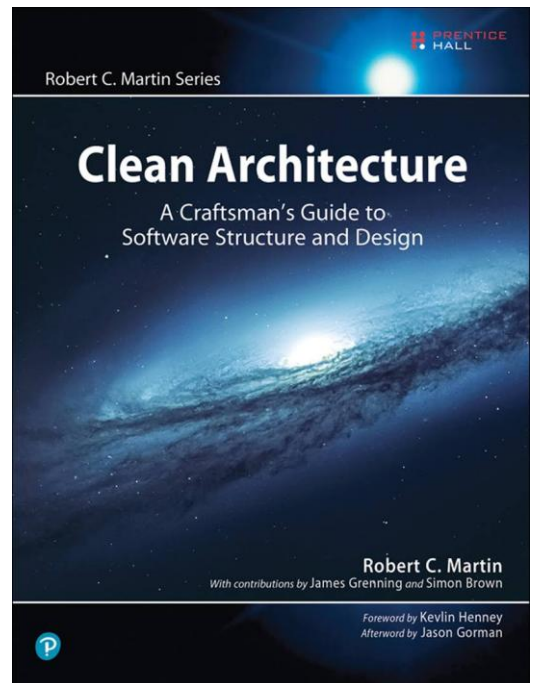
osebe.sort((prva, druga) -> prva.getStarost() - druga.getStarost());

System.out.println(osebe);

List<Oseba> mladoletne = osebe.stream().filter(nekdo -> nekdo.getStarost() < 18).collect(Collectors.toList());
System.out.println(mladoletne);
```

53

Čez poletje



54